

Aula 17

DataPrev (Perfil 3: Desenvolvimento de Software) Desenvolvimento de Software - 2026 (Pós-Edital)

Autor:
Vinicius Borges

09 de Julho de 2026

Índice

1) JSON - Teoria	3
2) JSON - Questões Comentadas	15
3) JSON - Lista de Questões	30

JSON

Conceitos Gerais

JSON, ou **JavaScript Object Notation**, é um formato de arquivo em texto destinado a carregar dados através de comunicações entre sistemas. Apesar de contar com JavaScript no nome, o JSON é um formato de arquivo que independe da linguagem implementada, sendo usado e aceito pela maior parte das linguagens de programação.

O nome "JavaScript Object Notation" indica a forma de estruturação do arquivo. Sua sintaxe é completamente baseada na sintaxe dos objetos de JavaScript. Portanto, temos pares de atributo e valor, separados por vírgulas e envoltos em chaves {}. Então, a sintaxe geral de um arquivo JSON se dá da seguinte forma:

```
{  
  atributo1:valor1,  
  atributo2:valor2,  
  ...  
  atributon: valorn  
}
```

QUESTÃO DE PROVA



(CEBRASPE/CNPq/2024) No que se refere a serviços de integração, julgue o item que se segue.

A sequência a seguir é uma lista que representa um objeto no formato JSON.

{atributo1:valor1 , atributo2:valor 2}

Comentários:

Arquivos JSON são conjuntos de dados em atributo:valor, envoltos de chaves {}. (Gabarito: Certo)

Alguns atributos dos arquivos JSON o levam a ser o formato preponderante na troca de dados na internet nos dias atuais, principalmente se comparados com um outro formato bem adotado - o XML. Veja:

- Legibilidade: maior legibilidade para humanos
- Baixa verbosidade: a ausência de tags para cada elemento o torna um arquivo menos verboso e, conseqüentemente, mais leve
- Velocidade de processamento: o JSON tem uma velocidade de processamento maior, se comparado aos seus pares, como o XML
- Compatibilidade: a maior parte das APIs modernas suporta o formato JSON
- Integração com JavaScript: por ser nativo do JavaScript, vários métodos da linguagem fazem uma integração completa com o formato

Abaixo, vou deixar um exemplo de arquivo JSON - e você perceberá como é tranquila a leitura.

```
{
  "nome": "João Silva",
  "idade": 30,
  "sexo": "Masculino",
  "email": "joao.silva@example.com",
  "telefone": "+55 11 98765-4321",
  "endereco": {
    "rua": "Rua das Flores",
    "numero": 123,
    "bairro": "Centro",
    "cidade": "São Paulo",
    "estado": "SP",
    "cep": "01234-567"
  },
  "interesses": ["Viajar", "Tecnologia", "Esportes"],
  "educacao": [
    {
      "instituicao": "Universidade XYZ",
      "curso": "Engenharia de Software",
      "ano_conclusao": 2015
    },
    {
      "instituicao": "Escola ABC",
      "curso": "Ensino Médio",
      "ano_conclusao": 2010
    }
  ]
}
```

Sintaxe

A sintaxe dos arquivos JSON são bem simples:

- Dado são armazenado em pares de chave:valor
- Cada dado deve ser separado por vírgula
- Pares de colchetes indicam *arrays*
- Pares de chaves indicam um objeto

Ao contrário dos objetos em JavaScript, **nos arquivos JSON precisamos ter as chaves**, que definem os valores, **como strings** - portanto, envoltas em aspas duplas “. Então, enquanto uma linha de código de objeto em JavaScript pode ser escrita como nome: “Felipe Mathias”, em JSON ela deve ser escrita como “nome”: “Felipe Mathias”.

Já para os **valores**, temos algumas definições diferentes. Podemos ter os seguintes tipos de dados, que seguem a mesma padronização de formato que o JavaScript:

- Texto (*string*). Ex.: {“nome”: “Estratégia”}
- Número (inteiro ou de ponto flutuante). Ex.: {“nota”: 7.39}
- Objeto. Ex.: {“nome”: “Desenvolvimento de Sistemas”, “Matéria”: “TI”}
- Array. Ex.: {“professores”: [“Felipe”, “Paolla”]}
- Boolean. Ex.: {“aprovado”: true}
- Null. Ex.: {“pendencias”: null}

QUESTÃO DE PROVA



Em se tratando da persistência de dados, os bancos de dados orientados a documentos se mostram adequados para representar e armazenar dados que possuem características comuns, mas que também possuem características distintas entre si. Um dos formatos muito utilizados para representação de dados em um banco de dados orientados a documentos é o formato Javascript Object Notation (JSON).

(CESGRANRIO/IPEA/2024) Nesse contexto, considere o exemplo em JSON apresentado a seguir, que representa dados de um livro da área de banco de dados:

{

```
{
  "isbn": "9788543025001",
  "autor": [
    {
      "ultimonome": "Elmasri",
      "primeironome": "Ramez"
    },
    {
      "ultimonome": "Navathe",
      "primeironome": "Shamkant"
    }
  ],
  "titulo": "Sistemas de Bancos de Dados",
  "categoria": ["BD", "SGBDR", "SQL"]
}
```

No exemplo apresentado, observa-se que

- a) o livro possui dois vetores, ou arrays.
- b) o livro tem cinco pares de nome e valor.
- c) o campo "categoria" tem três pares de nome e valor.
- d) o campo de nome "9788543025001" tem o valor "isbn".
- e) o campo de nome "autor" é formado por um vetor, ou array, de quatro elementos.

Comentários:

Vamos analisar cada alternativa.

- a) Certo. Temos duas arrays, caracterizadas pelos colchetes [] - uma para "autor", e outra para "titulo".
- b) Errado. Temos 4 pares - isbn, autor, titulo e categoria.
- c) Errado. O campo "categoria" é uma array, contendo 3 valores distintos.
- d) Errado. O campo de nome "isbn" que tem o valor numérico, e não o contrário.
- e) Errado. Ele é formado por uma array de 2 elementos, e cada elemento desses é um objeto composto de dois pares de chave-valor.

Portanto, correta a letra A. (Gabarito: Letra A)

Parse

O **Parse** do JSON, ou, numa tradução literal, sua “tradução”, implica em convertermos o formato de texto para um formato legível e utilizável pelas linguagens. Especificamente para o JavaScript, como o JSON é um formato de arquivo nativo, temos uma função completamente integrada que permite tanto transformar JSON em objetos, quanto o caminho oposto.

Para isso, usamos a função **JSON.parse()**, em JavaScript. Por exemplo, imagine o seguinte conjunto de dados em JSON:

```
{  
  "nome": "Felipe Mathias"  
  "profissão": "Professor"  
  "idade": 30  
}
```

Se quisermos manipular esses dados em JavaScript, selecionando somente uma chave, por exemplo, podemos fazer da seguinte forma:

```
jQuery  
  
const jsonObj = JSON.parse('{ "nome": "Felipe Mathias", "profissão":  
"Professor", "idade": 30 }');  
  
console.log("Nome: ", jsonObj["nome"]);  
  
Nome: Felipe Mathias
```

Agora pense essa fórmula aplicada a uma requisição HTTP, retornando um conjunto de dados e permitindo que você navegue através desse arquivo JSON com métodos nativos do JavaScript, criando um código extremamente adaptável e de baixo acoplamento.

INDO MAIS FUNDO!



Também é possível que façamos o caminho oposto. A partir de um objeto qualquer, criamos uma *string* no formato de um arquivo JSON com a função `JSON.stringify()`. Vamos fazer o mesmo que fizemos anteriormente, mas agora no caminho oposto - começaremos com um objeto qualquer e o converteremos em um arquivo JSON.

```
jQuery

const jsonObj = {
  nome: "Felipe Mathias",
  profissao: "Professor",
  idade: 30
}

let jsonString = JSON.stringify(jsonObj)

console.log("O tipo do dado é :", typeof jsonString)
console.log(jsonString)

O tipo do dado é : string
{"nome":"Felipe Mathias","profissao":"Professor","idade":30}
```

Por fim, podemos fazer uso da função `eval`, do JavaScript, para converter um texto JSON em um objeto no JavaScript, principalmente em navegadores que não suportam o `JSON.parse`. A função `eval()` em JavaScript é usada para avaliar ou executar uma string como código JavaScript. Nesse caso, criamos uma string com base no JSON, e a transformamos em um objeto com a função `eval()`. O código ficará da seguinte forma:

```
jQuery

// JSON em formato de string
const jsonString = '{"nome": "Felipe Mathias", "profissão": "Professor", "idade": 30}';

// Usando eval para converter JSON em um objeto JavaScript
const jsonObject = eval('(' + jsonString + ')');

console.log(jsonObject.nome);

Felipe Mathias
```

ESTA É DIFÍCIL!



(CEBRASPE/TCE SC/2016) Julgue o item que se segue a respeito dos padrões XSLT e JSON.

Em navegadores que não possuem apoio para a função JavaScript `JSON.parse`, pode-se utilizar a função `eval` para converter um texto JSON em um objeto JavaScript, por meio da sintaxe apresentada a seguir.

```
var obj = eval("(" + text + ");");
```

Comentários:

Perfeito, aluno. O `eval()` é uma das formas de transformarmos um arquivo JSON, que é interpretado como uma string, em um objeto dentro do JavaScript, permitindo a aplicação de métodos de manipulação. (Gabarito: Certo)

(Inédita/Prof. Felipe Mathias) Assinale a alternativa que aponta o comando a ser utilizado para converter um arquivo JSON em um objeto que pode ser manipulado pelo JavaScript:

- a) `JSON.purge`
- b) `JSON.pause`
- c) `JSON.convert`
- d) `JSON.parse`
- e) `JSON.object`

Comentários:

Ao recebermos um arquivo JSON, que estará em formato de texto (string), é necessário transformá-lo em objeto para corretamente manipular suas propriedades, acessar suas chaves, entre outros. Para isso, usa-se o `JSON.parse`. (Gabarito: Letra D)

Dados tabulares

Muitas vezes utilizamos consultas em SQL para criarmos os arquivos JSON. Precisamos tratar os dados retornados, que são tabulares, em um arquivo JSON para ser utilizado pelos códigos. O JSON não traz nenhuma forma de especificação de estrutura para dados tabulares e, portanto, usualmente tratamos esses dados com um conjunto de arrays, sendo:

- 1 Array para cada atributo (coluna)
- 1 Array composto de outras arrays, representando cada linha isoladamente

Pode ter parecido confuso, então vou lhe trazer um exemplo.



Imagine a seguinte tabela:

ID	Nome	Idade	País
1	João	30	Brasil
2	Maria	25	EUA
3	Carlos	40	Espanha

Representando esses dados em JSON, teríamos a seguinte estrutura:

```
{  
  "dados": {  
    "colunas": ["ID", "Nome", "Idade", "País"],  
    "linhas": [  
      [1, "João", 30, "Brasil"],  
      [2, "Maria", 25, "EUA"],  
      [3, "Carlos", 40, "Espanha"]  
    ]  
  }  
}
```

Comentários

Comentários na grande maioria das linguagens de programação são pequenas linhas de instrução, destinadas ao desenvolvedor que irá ler o código, e não ao compilador - e, por esse motivo, elas não são interpretadas. Porém, o JSON não é uma linguagem de programação, e sim uma simples linguagem em texto para transporte de dados.

Por esse motivo, **os comentários em JSON não são aceitos**. Se inserido algum comentário, usualmente por uma simbologia de destaque (como uma cerquilha #), o compilador irá os interpretar como os dados, eles não serão ignorados.

A alternativa mais recomendada é a criação de um par de chave-valor para servir de comentário. Então, a cada novo elemento, podemos criar uma variável por exemplo `"_comentario": "linha de comentario"`.

QUESTÃO DE PROVA



(CEBRASPE/PREF. FORTALEZA/2023) Acerca de HTTPRequest e JSON, julgue o próximo item.

O processo de adição de comentários em arquivos JSON é igual ao das linguagens de programação, pois, assim como estes, tipicamente ignora comentários quando são executados.

Comentários:

O JSON não suporta comentários "tradicionais". Uma forma de sobrepor isso é criando pares de chave-valor que são facilmente identificados como comentários - mas, apesar disso, essa linha ainda será interpretada pelo compilador. Dessa forma, a alternativa está incorreta. (Gabarito: Errado)

JSON Schema

Os arquivos JSON não possuem um formato pré-definido, por isso são categorizados nos dados de tipo semiestruturado. Porém, para determinados sistemas, pode ser necessário que um padrão seja seguido, garantindo o correto funcionamento do arquivo.

Para fazer uma validação do esquema, utilizamos o **JSON Schema**. O arquivo JSON recebido (ou a ser enviado) é passado por um validador, que utiliza o esquema definido para verificar se o arquivo está de acordo com as diretrizes. Esse esquema tem diversos padrões - adotaremos o padrão do json-schema.org para essa aula.

O arquivo Schema parece-se com isso:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Book",
  "type": "object",
  "properties": {
    "title": {
      "type": "string",
      "description": "The title of the book"
    },
    "author": {
      "type": "string",
      "description": "The author of the book"
    },
    "published_year": {
      "type": "integer",
      "minimum": 1900,
      "maximum": 2024,
      "description": "The year the book was published"
    },
    "isbn": {
      "type": "string",
      "pattern": "^\\d{3}-\\d{10}$",
      "description": "The ISBN of the book in the format xxx-xxxxxxxxxx"
    }
  },
  "required": ["title", "author", "published_year"],
  "additionalProperties": false
}
```

- **\$schema**: Especifica a versão do JSON Schema utilizada.
- **title**: Define o título do esquema.
- **type**: Indica que o objeto JSON esperado deve ser do tipo "object".
- **properties**: Define as propriedades esperadas no objeto JSON, como "title", "author", "published_year" e "isbn".

- Para cada propriedade, são especificados o tipo (type), uma descrição (description) e, quando aplicável, restrições adicionais, como o ano mínimo e máximo de publicação para "published_year" e um padrão de regex para "isbn".
- **required:** Lista as propriedades que são obrigatórias no objeto JSON.
- **additionalProperties:** Define se propriedades adicionais não especificadas no esquema são permitidas (true) ou não (false). Neste caso, propriedades adicionais não são permitidas.

QUESTÃO DE PROVA



(Inédita/Prof. Felipe Mathias) Acerca dos conhecimentos sobre as tecnologias Web, julgue o item abaixo.

É possível criar uma estrutura; ao mais forte aos arquivos JSON, exigindo sua validação, a partir do uso do Esquema JSON, ou JSON Schema.

Comentários:

Certinho! O JSON Schema é responsável por definir alguns padrões para o arquivo JSON, como tipo de dado, tamanho, tipo de informação e campos obrigatórios. (Gabarito: Certo)

RESUMO

O QUE É O JSON?

JSON, ou JavaScript Object Notation, é um **arquivo de texto destinado a carregar informações e dados entre sistemas distintos**. Ele é um arquivo independente de implementação, aceitado em diversas linguagens, apesar de ser nativo do JavaScript.

COMO É A NOTAÇÃO DO JSON?

No JSON, temos uma notação baseada nos objetos do JavaScript, com algumas alterações. Por ser uma string, envolvemos todos os dados num par de chaves {}. Os dados, por sua vez, são representados em pares de "chaves": "valor". Então, por exemplo, podemos ter a notação {"aula": "JSON"}.

QUESTÕES COMENTADAS

01. (CESGRANRIO/IPEA/2024) Um programador de páginas web escreveu o arquivo XML abaixo no formato JSON, para que usuários internos do Ipea possam gerenciar o acesso aos dados públicos das páginas web.

```
<menu id="file" value="File">
  <popup>
    <menuitem value="Localizar" onclick="Search()" />
    <menuitem value="Alterar" onclick="Alter()" />
    <menuitem value="Incluir" onclick="Include()" />
    <menuitem value="Apagar" onclick="Delete()" />
  </popup>
</menu>
```

Após analisar o arquivo acima, o gerente da área corrigiu a sua sintaxe, obtendo o seguinte arquivo, no formato JSON:

a)

```
{ "menu": {
  "id": "file",
  "value": "File",
  "popup": {
    [
      { "menuitem": "Localizar", "onclick": "Search()" },
      { "menuitem": "Alterar", "onclick": "Alter()" },
      { "menuitem": "Incluir", "onclick": "Include()" },
      { "menuitem": "Apagar", "onclick": "Delete()" }
    ]
  }
}}
```

b)

```
{ "menu": { BEGIN
  { "id": "file", "value": "File" }
  "popup": {
    "menuitem": [
      { "Localizar", "onclick": "Search()" },
      { "Alterar", "onclick": "Alter()" },
      { "Incluir", "onclick": "Include()" },
      { "Apagar", "onclick": "Delete()" }
    ]
  }
} END }
```

c)

```
{ BEGIN "menu"
  "id": "file",
```

```
"value": "File",  
"popup": {  
  "menuitem": [  
    {"value": "Localizar", "Search()"},  
    {"value": "Alterar", "Alter()"},  
    {"value": "Incluir", "Include()"},  
    {"value": "Apagar", "Delete()"}  
  ]  
}  
}  
END}
```

```
d)  
{'menu': {  
  "id": "file",  
  "value": "File",  
  "popup": {  
    "menuitem": [  
      {"value": "Localizar", "onclick": "Search()"},  
      {"value": "Alterar", "onclick": "Alter()"},  
      {"value": "Incluir", "onclick": "Include()"},  
      {"value": "Apagar", "onclick": "Delete()"}  
    ]  
  }  
}  
}}
```

```
e)  
{'menu': {  
  "id": "file",  
  "value": "File",  
  "popup": {  
    "menuitem": [  
      {"item": "Localizar", "Search()"},  
      {"item": "Alterar", "Alter()"},  
      {"item": "Incluir", "Include()"},  
      {"item": "Apagar", "Delete()"}  
    ]  
  }  
}  
}}
```

Comentários:

Questão complexa. Vamos fazer a conversão por partes, para entendermos o que acontecerá.

```
<menu id="file" value="File">
```

Esse trecho aponta que temos uma *tag* de nome menu, com id file e valor File. Podemos traduzir a sintaxe para um objeto, chamado de “menu”.

```
{ "menu": {  
  "id": "file",  
  "value": "file",
```

Vamos à segunda parte.

```
<popup>  
  <menuitem value="Localizar" onclick="Search()" />  
  <menuitem value="Alterar" onclick="Alter()" />  
  <menuitem value="Incluir" onclick="Include()" />  
  <menuitem value="Apagar" onclick="Delete()" />  
</popup>
```

Temos um *popup* com diversos itens de menu, cada um com seu efeito *onclick* e seu valor. Vamos representar o popup como um objeto, e cada menuitem como um item de uma array de objetos:

```
{ "popup": {  
  "menuitem": [  
    { "value": "Localizar", "onclick": "Search()" },  
    { "value": "Alterar", "onclick": "Alter()" },  
    { "value": "Incluir", "onclick": "Include()" },  
    { "value": "Apagar", "onclick": "Delete()" }  
  ]  
}
```

Juntando as duas sintaxes, chegamos à seguinte opção:

```
{ "menu": {  
  "id": "file",  
  "value": "File",  
  "popup": {  
    "menuitem": [  
      { "value": "Localizar", "onclick": "Search()" },  
      { "value": "Alterar", "onclick": "Alter()" },  
      { "value": "Incluir", "onclick": "Include()" },  
      { "value": "Apagar", "onclick": "Delete()" }  
    ]  
  }  
}
```

Gabarito: Letra D

02. (CEBRASPE/ITAIPU/2024) Assinale a opção que apresenta a sintaxe correta de representação de um array de nomes e sobrenomes de duas pessoas na linguagem JSON.

- a) { { "nome": "JOAO", "sobrenome": "SILVA" } , { "nome": "MARIA", "sobrenome": "SOARES" } }
- b) [{ "nome": "JOAO", "sobrenome": "SILVA" } { "nome": "MARIA", "sobrenome": "SOARES" }]
- c) { "nome": "JOAO", "sobrenome": "SILVA", "nome": "MARIA", "sobrenome": "SOARES" }
- d) [{ "nome": "JOAO", "sobrenome": "SILVA" } , { "nome": "MARIA", "sobrenome": "SOARES" }]

e) { ("nome": "JOAO", "sobrenome": "SILVA") , ("nome": "MARIA", "sobrenome": "SOARES") }

Comentários:

A representação de arrays em JSON é feita a partir de um par de colchetes []. Podemos representar os nomes, portanto, como:

```
{ "nomes":  
  [ {"nome": "nome1", "sobrenome": "sobrenome1"},  
    {"nome": "nome 2", "sobrenome": "sobrenome2"} ]  
}
```

Portanto, a alternativa correta é a letra D.

Gabarito: Letra D

03. (CEBRASPE/SEPLAN RR/2023) Com relação a programação e desenvolvimento de sistemas, julgue o item a seguir.

A seguinte notação em JSON representa corretamente a propriedade de "Nome" para as empresas A e B.

```
{"Empresas":[{"Nome":"A"}, {"Nome":"B"}]}
```

Comentários:

Quando vamos elencar atributos compostos e de "mesmo nível", como é o caso das empresas, podemos fazer isso através de uma array, representada pelos colchetes []. Para cada empresa criaremos um objeto dentro dessa array, que receberá suas características. É exatamente isso que a questão fez - portanto, correta.

Gabarito: Correto

04. (FGV/TCE ES/2023) Pedro está desenvolvendo um programa que manipula dados do tipo Arrays (Vetores) no formato JSON.

Para representar dados desse tipo, Pedro deve usar a sintaxe:

- a) {"venda":true}
- b) {"nome":"John"}
- c) {"idade":null}
- d) {"cadastro":["Joao", "Ana", "Pedro"]}
- e) {"pessoa":{"nome":"Joao", "idade":30, "estado":"ES"}}

Comentários:

As arrays em JSON são representadas a partir de um par de colchetes []. A alternativa que apresenta a sintaxe é a letra D - que cria uma array para a chave "cadastro".

05. (IADES/SEPLAD DF/2023) Qual é o caractere que delimita um array em uma estrutura JSON?

- a) Parênteses
- b) Colchetes
- c) Chaves
- d) Asteriscos
- e) Cifrões

Comentários:

Você já deve ter percebido que a cobrança vem pesada nas *arrays* em JSON, né? Lembre-se, as arrays são representadas por pares de colchetes [].

06. (VUNESP/TCM SP/2023) A alternativa que contém uma representação inválida de um objeto JSON, segundo o padrão ECMA-404, é:

- a) {"op": "test", "path": "/a/b/c", "value": "foo"}
- b) {"op": "replace", "path": "/a/b/c", "value": 42}
- c) {"op": "add", "path": "/a/b/c", "value": ["foo", "bar"]}
- d) {"op": "move", "from": "/a/b/c", "path": "/a/b/d"}
- e) {"op": "copy", "path", "/a/b/c", "value": "foo"}

Comentários:

As sintaxes, com exceção da letra E, estão todas corretas. Na letra D, o correto seria "path" : "/a/b/c", e não o uso de vírgula, como foi feito.

07. (FUMARC/ALMG/2023) Analise as afirmativas a seguir referentes ao formato JSON:

- I – JSON é um acrônimo de JavaScript Object Notation.
- II – JSON é um formato para intercâmbio de dados.
- III – Um objeto JSON começa com uma "{" e termina com uma "}".

Estão CORRETAS as afirmativas:

- a) I e II, apenas.
- b) I e III, apenas.
- c) II e III, apenas.
- d) I, II e III.

Comentários:

Vamos analisar os itens.

- I. Certo. Esse é o significado de JSON.
- II. Certo. O JSON é destinado a intercâmbio de dados entre sistemas.
- III. Certo. Identificamos arquivos JSON pela presença de pares de chaves {}.

Portanto, todos os itens são corretos.

Gabarito: Letra D

08. (FCC/MPE PB/2023) Considere as propriedades abaixo, seus valores e tipos de dados.

- id, tipo numérico, contendo o valor 432.
- nome, cadeia de caracteres, contendo o valor Paulo.
- cargo, cadeia de caracteres, contendo o valor Promotor.

A forma adequada de criar um objeto JSON contendo estes valores é

- a) ["id":432, "nome":"Paulo", "cargo":"Promotor"]
- b) {number["id":432], String["nome":"Paulo", "cargo":"Promotor"]}
- c) {id[number]:432, nome[string]:"Paulo", cargo[string]:"Promotor"}
- d) [int id=432, String nome="Paulo", String cargo="Promotor"]
- e) {"id":432, "nome":"Paulo", "cargo":"Promotor"}

Comentários:

Vamos converter cada linha.

- id, tipo numérico, contendo o valor 432.
{"id" : 432}

- nome, cadeia de caracteres, contendo o valor Paulo.
{"nome" : "Paulo"}

- cargo, cadeia de caracteres, contendo o valor Promotor.
{"cargo" : "Promotor"}

A alternativa que, corretamente, traz todas as conversões é a letra E.

Gabarito: Letra D

09. (FCC/MPE PB/2023) Considere o fragmento JavaScript abaixo.

```
<script>  
const json_adv = '___|___';  
const obj_adv = JSON.parse(json_adv);  
</script>
```

Na lacuna |, a representação correta para um objeto JSON contendo a propriedade nome com o valor Paulo e um array processos contendo os processos 0001842672017 e 0045613912014 é

- a) {"nome":"Paulo", "processos[0]":"0001842672017", "processos[1]":"0045613912014"}
- b) {nome="Paulo", processos=["0001842672017", "0045613912014"]}
- c) {"nome":"Paulo", "processos":["0001842672017", "0045613912014"]}
- d) nome="Paulo", processos{"0001842672017", "0045613912014"}
- e) [{"nome":"Paulo", "processos":{"0001842672017", "0045613912014"}}]

Comentários:

Podemos representar o nome como {"nome":"Paulo"}, e a *string* como "processos": ["00018...", "00456..."]. A alternativa que corretamente faz essas conversões é a letra C.

Gabarito: Letra D

10. (FUNDATEC/BRDE/2023) Sobre o JSON (JavaScript Object Notation), assinale a alternativa que apresenta um tipo de dado não suportado.

- a) date
- b) string
- c) number
- d) boolean
- e) array

Comentários:

Dos tipos apresentados, aquele não suportado pelo JSON está na letra A - o formato DATE. Temos um conjunto de tipos de dado usados no JavaScript que **não são suportados** pelo JSON:

- Date
- Função
- Undefined

Portanto, nosso gabarito é a letra A.

Gabarito: Letra A

11. (FEPESE/PREF. B. CAMBORIÚ/2023) São tipos de dados JSON válidos:

- 1. Null
- 2. Data
- 3. Array
- 4. Undefined
- 5. Booleano

Assinale a alternativa que indica todas as afirmativas corretas.

- a) São corretas apenas as afirmativas 2 e 4.
- b) São corretas apenas as afirmativas 3 e 5.

- c) São corretas apenas as afirmativas 1, 2 e 3.
- d) São corretas apenas as afirmativas 1, 3 e 5.
- e) São corretas apenas as afirmativas 3, 4 e 5.

Comentários:

Vamos listar os tipos de dados permitidos e vedados?

Tipos de Dados JSON	
Permitidos	Vedados
String	Função
Números (int e float)	Date
Objetos	Undefined
Array	
Boolean	
Null	

Analisando a lista, os tipos de dados permitidos são o 1, 3 e 5, enquanto 2 e 4 são vedados.

Gabarito: Letra D

12. (FUNDATEC/PROCERGS/2023) O formato JSON (JavaScript Object Notation) surgiu em 2000 como uma alternativa ao formato XML (eXtensible Markup Language). Apresenta-se como um formato de intercâmbio de dados mais leve e com maior facilidade de interpretação para uma boa comunicação de aplicações web. Sendo assim, são características do formato JSON:

- I. Suporta uma variedade de tipos de dados, tais como números, strings, booleanos, arrays e objetos.

- II. É um formato dependente, sendo utilizado em aplicações web apenas com a linguagem de programação Javascript.
- III. É um formato mais leve que o XML, o que significa que pode ser transmitido com maior rapidez.
- IV. Tem uma estrutura definida com tags que delimitam os elementos de dados e atributos que fornecem informações adicionais sobre esses elementos.

Quais estão INCORRETAS?

- a) Apenas I.
- b) Apenas III.
- c) Apenas II e IV.
- d) Apenas I, II, III.
- e) I, II, III e IV.

Comentários:

Vamos às alternativas.

- I. Certo. Temos diversos tipos diferentes de dados suportados.
- II. Errado. O JSON é independente de implementação.
- III. Certo. O JSON, por ser menos verboso, é mais leve e trafega de forma mais rápida que o XML.
- IV. Errado. A estrutura do JSON não é definida.

Portanto, temos II e IV como incorretas.

Gabarito: Letra C

13. (VUNESP/TJ RS/2023) Uma API RESTful de um sistema de uma loja de livros, quando acessada pela requisição

GET /api/v1/livros

retorna a seguinte resposta no formato JSON:

```
{
  "meta": {
  },
  "data": [
    {
      "id": 20,
      "title": "Mensagem",
      "author": "Fernando Pessoa"
    },
    {
      "id": 21,
      "title": "Alguma Poesia"
    }
  ]
}
```

```
"author : Carlos Drummond de Andrade  
  }  
}
```

Com base nessas informações, é correto afirmar que

- a) meta é um array vazio.
- b) o conteúdo retornado está sintaticamente incorreto de acordo com o padrão ECMA-404, pois faltam aspas nos valores de id.
- c) o array data possui 2 elementos.
- d) o array data possui 6 elementos.
- e) o código de status HTTP retornado como resposta é 302 – Found.

Comentários:

Vamos analisar cada alternativa.

- a) Errado. A chave "meta" é um objeto, não uma array.
- b) Errado. Não há necessidade de aspas para especificar valores.
- c) Certo. O array "data" é composto de 2 objetos.
- d) Errado. Cuidado para não confundir - o array tem 2 elementos, 2 objetos.
- e) Errado. Um pouco fora do escopo da questão, mas 302 é um código de erro, para página temporariamente mudada.

Gabarito: Letra C

14. (CEBRASPE/MPE RO/2023) Em ambiente web, o padrão que é utilizado para a troca de informações entre sistemas e que apresenta estrutura composta por elementos do tipo chaves, dois pontos, colchetes e aspas é o

- a) SOAP.
- b) XML.
- c) JSON.
- d) XSLT.
- e) WSDL.

Comentários:

O formato de arquivo com uma estrutura composta por um par de chaves é o JSON.

Gabarito: Letra C

15. (VUNESP/UFABC/2023) Na notação JSON, de acordo com a especificação ECMA 404, a sequência \u000d, quando usada dentro de uma string, representa o caractere

- a) backspace.
- b) barra normal.

- c) barra invertida.
- d) retorno de carro (carriage return).
- e) alimentação de linha (line feed).

Comentários:

Questão bem avançada. O caractere `\u000d` indica o caractere *retorno de carro* (*carriage return*). Esse caractere indica um fim de linha de texto para os sistemas operacionais.

Gabarito: Letra D

16. (FUVEST/USP/2023) Observe a notação JSON apresentada.

```
[  
  {"year": 2024},  
  {"location": "unknown"}  
]
```

Que tipo de dados está representado pelo componente mais externo da notação?

- a) Object.
- b) Array.
- c) Number.
- d) String.
- e) Dictionary.

Comentários:

O componente mais externo é um par de colchetes - que indica um Array.

Gabarito: Letra B

17. (CEBRASPE/TC DF/2023) A respeito de interoperabilidade de sistemas, DevOps e configuração de software, julgue o item que se segue.

Na notação JSON, cada objeto é representado por uma dupla de nome e valor de propriedade, agrupados entre chaves.

Comentários:

Perfeito! Objetos em JSON são representados por pares de chaves - por exemplo: `{"valores": {32, 34, 45, 92}}`.

Gabarito: Correto

18. (FGV/TJ SE/2023) JSON é um formato de texto autodescritivo utilizado para armazenar e transportar dados escritos em pares chave-valor. Para escrever um par chave-valor JSON, deve-se usar o formato:

- a) nome;Maria
- b) (nome:"Maria")
- c) {nome: Maria}
- d) [nome:'Maria']
- e) "nome": "Maria"

Comentários:

Para escrever um par de chave-valor em JSON, utilizamos "chave": "valor". Como a questão quer o par nome e Maria, escreveremos "nome": "Maria".

Gabarito: Letra E

19. (FGV/TJ SE/2023) A analista Deise recebeu a tarefa de mapear uma tabela de banco de dados para um arquivo no formato JavaScript Object Notation (JSON). Após analisar a estrutura da tabela, Deise concluiu que a melhor estratégia de mapeamento consiste em associar cada valor da tabela a um dos três nomes literais previstos como valores válidos na notação JSON. Ao executar essa estratégia, Deise deve associar cada valor da tabela a um dos nomes literais:

- a) true, false, null;
- b) True, False, Null;
- c) True, False, Undefined;
- d) infinite, nan, undefined;
- e) Infinite, NaN, Undefined.

Comentários:

O JSON aceita três valores literais: os valores booleanos (true e false) e null. Esses valores **devem** ser escritos em caixa baixa (minúsculas). A alternativa que traz corretamente esses três valores é a letra A.

Gabarito: Letra A

20. (FGV/TJ SE/2023) A técnica Joana é responsável pela manutenção do arquivo processos.json, que possui o seguinte conteúdo no formato JavaScript Object Notation (JSON):

```
{ "processosPorMunicipio": [  
  { "nome": "Pedrinhas", "qtd": 33144},  
  { "nome": "Telha", "qtd": 48958}]}
```

Joana deve adicionar ao array processosPorMunicipio um novo objeto, relacionado ao Município de Riachuelo, com os atributos nome e qtd. Porém, Joana ainda não dispõe da quantidade de processos de Riachuelo. Para adicionar o novo objeto sem omitir atributos, Joana deve utilizar o valor previsto na notação JSON para ausência de informação.

Logo, Joana deve definir o valor do atributo qtd do novo objeto como:

- a) 0;
- b) null;
- c) NaN;
- d) zero;
- e) undefined.

Comentários:

Como Joana não tem uma quantidade definida, a técnica deve adicionar um valor literal indicando o conjunto vazio - esse valor é representado pelo atributo null.

Gabarito: Letra B

21. (CEBRASPE/DP DF/2022) Julgue o item a seguir, acerca de CSS3, JMS, JSON e JUnit.

Datas devem ser escritas como strings para que sejam utilizadas em JSON.

Comentários:

Perfeito! Como não temos o formato DATE no JSON, é recomendado que as datas sejam escritas como conjuntos de caracteres (strings).

Gabarito: Correto

22. (CEBRASPE/DP DF/2022) Julgue o item a seguir, relativo a WSDL, JSON, XML e XSLT.

A estrutura a seguir descreve corretamente, na notação JSON, um veículo da marca ABC, modelo G5 e ano 2019. [marca:"ABC";modelo:"G5";ano:"2019"]

Comentários:

A array construída possui dois erros - primeiro ponto que deveria ser um objeto, e não uma array; o segundo é a ausência das aspas demarcando as chaves.

Gabarito: Errado

23. (FGV/PC AM/2022) Sobre as regras sintáticas do JSON (JavaScript Object Notation), avalie as seguintes afirmativas:

- I. Dados são codificados como pares name/value.
- II. Dados são separados por barras verticais "|".
- III. Chaves { } delimitam objetos.
- IV. Colchetes [] delimitam arrays.

Está correto somente o que é afirmado em

- a) I e II.
- b) II e III.
- c) III e IV.

- d) I, II e III.
- e) I, III e IV.

Comentários:

Vamos analisar os itens.

- I. Certo. Os dados no JSON são pares de chaves e valores.
- II. Errado. Os dados são separados por vírgulas.
- III. Certo. As chaves, além de delimitar o arquivo JSON, delimita também objetos.
- IV. Certo. Os colchetes [] são responsáveis por delimitar as arrays.

Temos, portanto, I, III e IV corretas.

Gabarito: Letra E

24. (FGV/SEFAZ AM/2022) Sobre a gramática da linguagem de intercâmbio de dados JSON (JavaScript Object Notation), assinale a afirmativa correta.

- a) Um objeto é um conjunto ordenado de pares nome-valor.
- b) Um valor numérico pode ser declarado em base decimal, octal ou hexadecimal.
- c) A ausência intencional de qualquer valor de objeto é representada com a palavra reservada *undefined*.
- d) Uma string é uma sequência de zero ou mais caracteres unicode envolto por aspas duplas.
- e) Os espaços em branco são proibidos entre qualquer par de tokens.

Comentários:

Vamos analisar cada alternativa.

- a) Errado. Os objetos são **desordenados**, isso é, não possuem uma ordem definida.
- b) Errado. Os valores numéricos só podem adotar a base decimal, seja inteiro ou float.
- c) Errado. A ausência de valores é representada por *null*.
- d) Certo. Essa é a definição de uma *string*.
- e) Errado. Os espaços em branco entre os tokens (chaves ou valores) são ignorados.

Gabarito: Letra D

25. (CEBRASPE/TCE RJ/2022) Julgue o item subsequente, a respeito dos padrões XML, XSLT, SOAP, REST e JSON.

JSON é uma alternativa ao XML para representar dados, baseado em texto e com pares nome e valor para representar as informações.

Comentários:

Perfeito! JSON é uma alternativa mais leve e menos verbosa ao XML, representando dados em pares de chaves e valores.

QUESTÕES COMENTADAS

01. (CESGRANRIO/IPEA/2024) Um programador de páginas web escreveu o arquivo XML abaixo no formato JSON, para que usuários internos do Ipea possam gerenciar o acesso aos dados públicos das páginas web.

```
<menu id="file" value="File">
  <popup>
    <menuitem value="Localizar" onclick="Search()" />
    <menuitem value="Alterar" onclick="Alter()" />
    <menuitem value="Incluir" onclick="Include()" />
    <menuitem value="Apagar" onclick="Delete()" />
  </popup>
</menu>
```

Após analisar o arquivo acima, o gerente da área corrigiu a sua sintaxe, obtendo o seguinte arquivo, no formato JSON:

a)

```
{ "menu": {
  "id": "file",
  "value": "File",
  "popup": {
    [
      { "menuitem": "Localizar", "onclick": "Search()" },
      { "menuitem": "Alterar", "onclick": "Alter()" },
      { "menuitem": "Incluir", "onclick": "Include()" },
      { "menuitem": "Apagar", "onclick": "Delete()" }
    ]
  }
}}
```

b)

```
{ "menu": { BEGIN
  { "id": "file", "value": "File" }
  "popup": {
    "menuitem": [
      { "Localizar", "onclick": "Search()" },
      { "Alterar", "onclick": "Alter()" },
      { "Incluir", "onclick": "Include()" },
      { "Apagar", "onclick": "Delete()" }
    ]
  }
} END }
```

c)

```
{ BEGIN "menu"
  "id": "file",
```

```
"value": "File",  
"popup": {  
  "menuitem": [  
    {"value": "Localizar", "Search()"},  
    {"value": "Alterar", "Alter()"},  
    {"value": "Incluir", "Include()"},  
    {"value": "Apagar", "Delete()"}  
  ]  
}  
}  
END}
```

```
d)  
{'menu': {  
  "id": "file",  
  "value": "File",  
  "popup": {  
    "menuitem": [  
      {"value": "Localizar", "onclick": "Search()"},  
      {"value": "Alterar", "onclick": "Alter()"},  
      {"value": "Incluir", "onclick": "Include()"},  
      {"value": "Apagar", "onclick": "Delete()"}  
    ]  
  }  
}  
}}
```

```
e)  
{'menu': {  
  "id": "file",  
  "value": "File",  
  "popup": {  
    "menuitem": [  
      {"item": "Localizar", "Search()"},  
      {"item": "Alterar", "Alter()"},  
      {"item": "Incluir", "Include()"},  
      {"item": "Apagar", "Delete()"}  
    ]  
  }  
}  
}}
```

02. (CEBRASPE/ITAIPU/2024) Assinale a opção que apresenta a sintaxe correta de representação de um array de nomes e sobrenomes de duas pessoas na linguagem JSON.

- a) { { "nome": "JOAO", "sobrenome": "SILVA" } , { "nome": "MARIA", "sobrenome": "SOARES" } }
- b) [{ "nome": "JOAO", "sobrenome": "SILVA" } { "nome": "MARIA", "sobrenome": "SOARES" }]
- c) { "nome": "JOAO", "sobrenome": "SILVA", "nome": "MARIA", "sobrenome": "SOARES" }

- d) [{ "nome": "JOAO", "sobrenome": "SILVA" } , { "nome": "MARIA", "sobrenome": "SOARES" }]
e) { ("nome": "JOAO", "sobrenome": "SILVA") , ("nome": "MARIA", "sobrenome": "SOARES") }

03. (CEBRASPE/SEPLAN RR/2023) Com relação a programação e desenvolvimento de sistemas, julgue o item a seguir.

A seguinte notação em JSON representa corretamente a propriedade de “Nome” para as empresas A e B.

`{"Empresas":[{"Nome":"A"}, {"Nome":"B"}]}`

04. (FGV/TCE ES/2023) Pedro está desenvolvendo um programa que manipula dados do tipo Arrays (Vetores) no formato JSON.

Para representar dados desse tipo, Pedro deve usar a sintaxe:

- a) `{"venda":true}`
b) `{"nome":"John"}`
c) `{"idade":null}`
d) `{"cadastro":["Joao", "Ana", "Pedro"]}`
e) `{"pessoa":{"nome":"Joao", "idade":30, "estado":"ES"}}`

05. (IADES/SEPLAD DF/2023) Qual é o caractere que delimita um array em uma estrutura JSON?

- a) Parênteses
b) Colchetes
c) Chaves
d) Asteriscos
e) Cifrões

06. (VUNESP/TCM SP/2023) A alternativa que contém uma representação inválida de um objeto JSON, segundo o padrão ECMA-404, é:

- a) `{"op": "test", "path": "/a/b/c", "value": "foo"}`
b) `{"op": "replace", "path": "/a/b/c", "value": 42}`
c) `{"op": "add", "path": "/a/b/c", "value": ["foo", "bar"]}`
d) `{"op": "move", "from": "/a/b/c", "path": "/a/b/d"}`
e) `{"op": "copy", "path", "/a/b/c", "value": "foo"}`

07. (FUMARC/ALMG/2023) Analise as afirmativas a seguir referentes ao formato JSON:

- I – JSON é um acrônimo de JavaScript Object Notation.
II – JSON é um formato para intercâmbio de dados.
III – Um objeto JSON começa com uma “{” e termina com uma “}”.

Estão CORRETAS as afirmativas:

- a) I e II, apenas.
- b) I e III, apenas.
- c) II e III, apenas.
- d) I, II e III.

08. (FCC/MPE PB/2023) Considere as propriedades abaixo, seus valores e tipos de dados.

- id, tipo numérico, contendo o valor 432.
- nome, cadeia de caracteres, contendo o valor Paulo.
- cargo, cadeia de caracteres, contendo o valor Promotor.

A forma adequada de criar um objeto JSON contendo estes valores é

- a) ["id":432, "nome":"Paulo", "cargo":"Promotor"]
- b) {number["id":432], String["nome":"Paulo", "cargo":"Promotor"]}
- c) {id[number]:432, nome[string]:"Paulo", cargo[string]:"Promotor"}
- d) [int id=432, String nome="Paulo", String cargo="Promotor"]
- e) {"id":432, "nome":"Paulo", "cargo":"Promotor"}

09. (FCC/MPE PB/2023) Considere o fragmento JavaScript abaixo.

```
<script>  
const json_adv = '___|___';  
const obj_adv = JSON.parse(json_adv);  
</script>
```

Na lacuna I, a representação correta para um objeto JSON contendo a propriedade nome com o valor Paulo e um array processos contendo os processos 0001842672017 e 0045613912014 é

- a) {"nome":"Paulo", "processos[0]":"0001842672017", "processos[1]":"0045613912014"}
- b) {nome="Paulo", processos=["0001842672017", "0045613912014"]}
- c) {"nome":"Paulo", "processos":["0001842672017", "0045613912014"]}
- d) nome="Paulo", processos{"0001842672017", "0045613912014"}
- e) ["nome":"Paulo", "processos":{"0001842672017", "0045613912014"}]

10. (FUNDATEC/BRDE/2023) Sobre o JSON (JavaScript Object Notation), assinale a alternativa que apresenta um tipo de dado não suportado.

- a) date
- b) string
- c) number
- d) boolean
- e) array

11. (FEPESE/PREF. B. CAMBORIÚ/2023) São tipos de dados JSON válidos:

1. Null
2. Data
3. Array
4. Undefined
5. Booleano

Assinale a alternativa que indica todas as afirmativas corretas.

- a) São corretas apenas as afirmativas 2 e 4.
- b) São corretas apenas as afirmativas 3 e 5.
- c) São corretas apenas as afirmativas 1, 2 e 3.
- d) São corretas apenas as afirmativas 1, 3 e 5.
- e) São corretas apenas as afirmativas 3, 4 e 5.

12. (FUNDATEC/PROCERGS/2023) O formato JSON (JavaScript Object Notation) surgiu em 2000 como uma alternativa ao formato XML (eXtensible Markup Language). Apresenta-se como um formato de intercâmbio de dados mais leve e com maior facilidade de interpretação para uma boa comunicação de aplicações web. Sendo assim, são características do formato JSON:

- I. Suporta uma variedade de tipos de dados, tais como números, strings, booleanos, arrays e objetos.
- II. É um formato dependente, sendo utilizado em aplicações web apenas com a linguagem de programação Javascript.
- III. É um formato mais leve que o XML, o que significa que pode ser transmitido com maior rapidez.
- IV. Tem uma estrutura definida com tags que delimitam os elementos de dados e atributos que fornecem informações adicionais sobre esses elementos.

Quais estão INCORRETAS?

- a) Apenas I.
- b) Apenas III.
- c) Apenas II e IV.
- d) Apenas I, II, III.
- e) I, II, III e IV.

13. (VUNESP/TJ RS/2023) Uma API RESTful de um sistema de uma loja de livros, quando acessada pela requisição

GET /api/v1/livros

retorna a seguinte resposta no formato JSON:

```
{  
  "meta": {  
  },  
  "data": [{
```

```
"id : 20,  
"title : Mensagem ,  
"author : Fernando Pessoa  
, {  
  "id : 21,  
  "title : Alguma Poesia ,  
  "author : Carlos Drummond de Andrade  
}  
}]  
}
```

Com base nessas informações, é correto afirmar que

- a) meta é um array vazio.
- b) o conteúdo retornado está sintaticamente incorreto de acordo com o padrão ECMA-404, pois faltam aspas nos valores de id.
- c) o array data possui 2 elementos.
- d) o array data possui 6 elementos.
- e) o código de status HTTP retornado como resposta é 302 – Found.

14. (CEBRASPE/MPE RO/2023) Em ambiente web, o padrão que é utilizado para a troca de informações entre sistemas e que apresenta estrutura composta por elementos do tipo chaves, dois pontos, colchetes e aspas é o

- a) SOAP.
- b) XML.
- c) JSON.
- d) XSLT.
- e) WSDL.

15. (VUNESP/UFABC/2023) Na notação JSON, de acordo com a especificação ECMA 404, a sequência \u000d, quando usada dentro de uma string, representa o caractere

- a) backspace.
- b) barra normal.
- c) barra invertida.
- d) retorno de carro (carriage return).
- e) alimentação de linha (line feed).

16. (FUVEST/USP/2023) Observe a notação JSON apresentada.

```
[  
  {"year": 2024},  
  {"location": "unknown"}  
]
```

Que tipo de dados está representado pelo componente mais externo da notação?

- a) Object.

- b) Array.
- c) Number.
- d) String.
- e) Dictionary.

17. (CEBRASPE/TC DF/2023) A respeito de interoperabilidade de sistemas, DevOps e configuração de software, julgue o item que se segue.

Na notação JSON, cada objeto é representado por uma dupla de nome e valor de propriedade, agrupados entre chaves.

18. (FGV/TJ SE/2023) JSON é um formato de texto autodescritivo utilizado para armazenar e transportar dados escritos em pares chave-valor. Para escrever um par chave-valor JSON, deve-se usar o formato:

- a) nome;Maria
- b) (nome:"Maria")
- c) {nome: Maria}
- d) [nome:'Maria']
- e) "nome": "Maria"

19. (FGV/TJ SE/2023) A analista Deise recebeu a tarefa de mapear uma tabela de banco de dados para um arquivo no formato JavaScript Object Notation (JSON). Após analisar a estrutura da tabela, Deise concluiu que a melhor estratégia de mapeamento consiste em associar cada valor da tabela a um dos três nomes literais previstos como valores válidos na notação JSON. Ao executar essa estratégia, Deise deve associar cada valor da tabela a um dos nomes literais:

- a) true, false, null;
- b) True, False, Null;
- c) True, False, Undefined;
- d) infinite, nan, undefined;
- e) Infinite, NaN, Undefined.

20. (FGV/TJ SE/2023) A técnica Joana é responsável pela manutenção do arquivo processos.json, que possui o seguinte conteúdo no formato JavaScript Object Notation (JSON):

```
{ "processosPorMunicipio": [  
  {"nome": "Pedrinhas", "qtd": 33144},  
  {"nome": "Telha", "qtd": 48958}]}
```

Joana deve adicionar ao array processosPorMunicipio um novo objeto, relacionado ao Município de Riachuelo, com os atributos nome e qtd. Porém, Joana ainda não dispõe da quantidade de processos de Riachuelo. Para adicionar o novo objeto sem omitir atributos, Joana deve utilizar o valor previsto na notação JSON para ausência de informação.

Logo, Joana deve definir o valor do atributo qtd do novo objeto como:

- a) 0;

- b) null;
- c) NaN;
- d) zero;
- e) undefined.

21. (CEBRASPE/DP DF/2022) Julgue o item a seguir, acerca de CSS3, JMS, JSON e JUnit.

Datas devem ser escritas como strings para que sejam utilizadas em JSON.

22. (CEBRASPE/DP DF/2022) Julgue o item a seguir, relativo a WSDL, JSON, XML e XSLT.

A estrutura a seguir descreve corretamente, na notação JSON, um veículo da marca ABC, modelo G5 e ano 2019. [marca:"ABC";modelo:"G5";ano:"2019"]

23. (FGV/PC AM/2022) Sobre as regras sintáticas do JSON (JavaScript Object Notation), avalie as seguintes afirmativas:

- I. Dados são codificados como pares name/value.
- II. Dados são separados por barras verticais "|".
- III. Chaves { } delimitam objetos.
- IV. Colchetes [] delimitam arrays.

Está correto somente o que é afirmado em

- a) I e II.
- b) II e III.
- c) III e IV.
- d) I, II e III.
- e) I, III e IV.

24. (FGV/SEFAZ AM/2022) Sobre a gramática da linguagem de intercâmbio de dados JSON (JavaScript Object Notation), assinale a afirmativa correta.

- a) Um objeto é um conjunto ordenado de pares nome-valor.
- b) Um valor numérico pode ser declarado em base decimal, octal ou hexadecimal.
- c) A ausência intencional de qualquer valor de objeto é representada com a palavra reservada undefined.
- d) Uma string é uma sequência de zero ou mais caracteres unicode envolto por aspas duplas.
- e) Os espaços em branco são proibidos entre qualquer par de tokens.

25. (CEBRASPE/TCE RJ/2022) Julgue o item subsequente, a respeito dos padrões XML, XSLT, SOAP, REST e JSON.

JSON é uma alternativa ao XML para representar dados, baseado em texto e com pares nome e valor para representar as informações.

GABARITO

GABARITO



- | | | |
|--------------|-------------|-------------|
| 1. Letra D | 9. Letra D | 17. Correto |
| 2. Letra D | 10. Letra A | 18. Letra E |
| 3. Correto | 11. Letra D | 19. Letra A |
| 4. Letra D | 12. Letra C | 20. Letra B |
| 5. Letra B | 13. Letra C | 21. Correto |
| 6. Letra E | 14. Letra C | 22. Errado |
| 7. Letra D | 15. Letra D | 23. Letra E |
| ▪ 8. Letra D | 16. Letra B | 24. Letra D |
| | | 25. Correto |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.